

day11笔记

一、面向对象

1、面向对象编程，分工协作，它的思想和python里面的面向对象编程是一样的

2、构造函数

- 可以通过函数构造一个对象出来
- 构造函数要求函数名称首字母大写，主要是为了和其他函数做区分
- 构造出来的对象，需要使用 `new` 关键字来调用

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Document</title>
7  </head>
8  <body>
9      <script>
10         // {} 创建出来的对象，多个对象之间的属性是重复的，有很多的冗余代码
11         // let person1 = {
12         //     name: '张三',
13         //     age: 18,
14         //     gender: '男'
15         // }
16         // let person2 = {
17         //     name: '王五',
18         //     age: 21,
19         //     gender: '男'
20         // }
21
22         // new Object()
23         // let person1 = new Object({
24         //     name: '张三',
25         //     age: 19
26         // })
27
28         // let person1 = new Object()
29         // person1.name = '王五'
30         // person1.age = 24
31         // console.log(person1)
32
33         // 工厂模式，像工厂里面的流水线一样，可以批量创建对象
34         // function person(name, age){
35         //     // 创建一个对象
36         //     let obj = {}
37         //     // 添加属性
38         //     obj.name = name
```

```

39      //      obj.age = age
40      //      // 返回给外界
41      //      return obj
42      //  }
43      // let p1 = person('张三', 18)
44      // let p2 = person('王五', 20)
45      // console.log(p1, p2)
46
47      // 构造函数 --- 就是工厂模式的升级版（简化操作）
48      function Person(name, age){
49          // 添加属性
50          this.name = name
51          this.age = age
52          // 添加方法
53          this.say = function(){
54              console.log(`${this.name}说: "你好啊! "~`)
55          }
56      }
57      let p1 = new Person('张三', 18)
58      let p2 = new Person('王五', 20)
59      console.log(p1, p2)
60      console.log(p1.name, p2.name)
61      console.log(p1['name'], p2['name'])
62
63      p1.say()
64      p2.say()
65  </script>
66 </body>
67 </html>

```

二、this对象

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Document</title>
7  </head>
8  <body>
9      <button>按钮1</button>
10     <button>按钮2</button>
11     <button>按钮3</button>
12     <script>
13         /*
14             this
15             + 指的是函数执行上下文（函数的执行环境）对象
16             + this是一个变化的对象，根据函数的调用方式不同，而指向不同的对象
17             + 函数调用的方式有多种形式
18             + 函数调用方式总结
19             => 函数名加括号调用 fn()

```

```

20         => 函数可以通过事件调用 btn.onclick = function(){ } || btn.onclick = ()=>{}
21         => 函数可以通过定时器调用 setInterval(()=>{}, 1000)
22         => 函数可以被对象调用
23         => ...
24     +
25     */
26
27     // 函数名加括号调用 --- 函数里面的this指向的是window对象（是js里面最大的对象，所有的内容
    都是在这个对象里面）
28     // function fn(){
29     //     console.log(this)
30     // }
31     // fn()
32
33     // 函数可以通过事件调用 --- 函数里面的this指向的是当前事件的调用者
34     // const btn = document.querySelector('button')
35     // btn.onclick = function(){
36     //     console.log(this)
37     // }
38
39     const aBtn = document.querySelectorAll('button')
40     // console.log(aBtn[0], aBtn[1], aBtn[2])
41
42     // 需求：当点击那个按钮的时候让其改变背景颜色
43     // aBtn[0].onclick = function(){
44     //     aBtn[0].style.backgroundColor = 'orange'
45     // }
46     // aBtn[1].onclick = function(){
47     //     aBtn[1].style.backgroundColor = 'orange'
48     // }
49     // aBtn[2].onclick = function(){
50     //     aBtn[2].style.backgroundColor = 'orange'
51     // }
52
53     // for(var i=0; i<aBtn.length; i++){
54     //     // for 循环是同步代码
55     //     // onclick 事件是异步代码
56     //     aBtn[i].onclick = function(){
57     //         this.style.backgroundColor = 'orange'
58     //     }
59     // }
60
61     // for(let i=0; i<aBtn.length; i++){
62     //     // for 循环是同步代码
63     //     // onclick 事件是异步代码
64     //     aBtn[i].onclick = function(){
65     //         aBtn[i].style.backgroundColor = 'orange'
66     //     }
67     // }
68
69     // 函数可以通过定时器调用 --- 函数里面的this指向的是window对象
70     // setInterval(function(){

```

```

71         //      console.log(this)
72         // }, 1000)
73
74         // 函数可以被对象调用 --- 当前的对象
75         let obj = {
76             name: '张三',
77             // say: function(){
78             //      console.log(this)
79             // }
80             say(){
81                 console.log(this)
82             }
83         }
84         obj.say()
85     </script>
86 </body>
87 </html>

```

三、class类

1、`class` 是在构造函数基础上改造而来的，提供了内置的语法

```

1  // 使用 class
2  class Person{
3      // 构造器，其实就是构造函数内部
4      // python def __init__()
5      constructor(name, age){
6          // 添加属性
7          this.name = name
8          this.age = age
9      }
10     // 添加方法
11     money(){
12         console.log(`${this.name}赚了个小目标!`)
13     }
14 }
15 const p1 = new Person('张三', 20)
16 const p2 = new Person('王五', 21)
17 console.log(p1, p2)
18 p1.money()
19 p2.money()

```

2、继承

```

1  // 父类
2  class Father{
3      constructor(name, age){
4          this.name = name
5          this.age = age
6      }
7      money(){

```

```

8     console.log(`赚了一个小目标! `)
9   }
10 }
11
12 // 子类
13 class Son extends Father{
14   constructor(name, age, gender){
15     // 先继承, 再扩展
16     super(name, age)
17     this.gender = gender
18   }
19 }
20 const s1 = new Son('张三', 18, '男')
21 console.log(s1)

```

四、选项卡效果

1、选项卡布局

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6    <title>Document</title>
7    <style>
8      *{
9        margin: 0;
10       padding: 0;
11     }
12     .tab{
13       width: 260px;
14       height: 420px;
15       border: 1px solid red;
16       margin: 50px auto;
17     }
18     .tab .btn{
19       width: 100%;
20       height: 50px;
21       background-color: orange;
22       display: flex;
23       /* 让子元素在父标记中水平对齐方式, 两边没有距离, 中间距离一样 */
24       justify-content: space-between;
25       /* justify-content: space-around; */
26       /* 让子元素在父标记中垂直居中 */
27       align-items: center;
28     }
29     .tab .btn button{
30       width: 23%;
31       height: 60%;
32     }

```

```

33     .tab .content{
34         width: 100%;
35         height: 370px;
36         background-color: hotpink;
37     }
38     .tab .content div{
39         width: 100%;
40         height: 100%;
41         text-align: center;
42         /* 让所有标记隐藏 */
43         display: none;
44     }
45 </style>
46 </head>
47 <body>
48     <div class="tab">
49         <div class="btn">
50             <button>电影</button>
51             <button>电视</button>
52             <button>小说</button>
53             <button>游戏</button>
54         </div>
55         <div class="content">
56             <!-- display: block; 可以让隐藏元素显示 -->
57             <div style="display: block;">电影模块</div>
58             <div>电视模块</div>
59             <div>小说模块</div>
60             <div>游戏模块</div>
61         </div>
62     </div>
63 </body>
64 </html>

```

2、选项卡逻辑

```

1  // js选择器封装
2  // all --- true 表示获取所有的, false 表示获取单个
3  function $(ele, all=false){
4      if(all){
5          return document.querySelectorAll(ele)
6      }else{
7          return document.querySelector(ele)
8      }
9  }
10
11 // 面向对象写法
12 class Tab{
13     constructor(){
14         // 获取元素
15         this.aBtn = $('.btn button', true)
16         this.aContent = $('.content div', true)

```

```

17 // 当实例化对象的时候调用
18 this.init()
19 }
20 // 初始化方法
21 init(){
22     this.switchBtn()
23 }
24 // 点击按钮切换背景颜色
25 // switchBtn(){
26 //     // 这个位置 this指向的是实例化这个对象
27 //     // console.log(this)
28
29 //     // this备份，在能找到this的地方先把它保存一份，后面在其他地方使用
30 //     let _this = this
31 //     for(let i=0; i<this.aBtn.length; i++){
32 //         this.aBtn[i].onclick = function(){
33 //             // 这个位置，在事件里面，this指向的是点击那个按钮，而遍历这里的this需要指向的是实例化对象
34 //             // console.log(this)
35 //             for(let j=0; j<_this.aBtn.length; j++){
36 //                 _this.aBtn[j].style.backgroundColor = ''
37 //             }
38 //             this.style.backgroundColor = 'skyblue'
39 //         }
40 //     }
41 // }
42
43 switchBtn(){
44     let _this = this
45     for(let i=0; i<this.aBtn.length; i++){
46         this.aBtn[i].onclick = function(){
47             _this.clearStyle()
48             this.style.backgroundColor = 'skyblue'
49             // _this.aBtn[i].style.backgroundColor = 'skyblue'
50             _this.aContent[i].style.display = 'block'
51         }
52     }
53 }
54 clearStyle(){
55     for(let i=0; i<this.aBtn.length; i++){
56         this.aBtn[i].style.backgroundColor = ''
57         this.aContent[i].style.display = 'none'
58     }
59 }
60 }
61 new Tab()

```

五、模块化

- 1、模块化就是一个单独的功能模块，可以解决命名冲突、可以防止文件之间有覆盖的情况
- 2、模块化

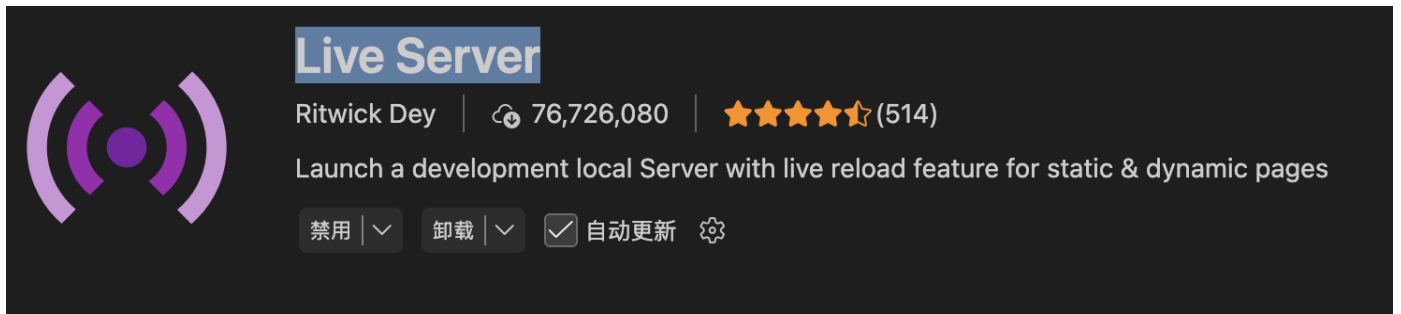
- 导出, `export`、`export default`
- 导入, `from`、`import`

3、注意点

- 使用 `script` 标记导入js文件时, 需要给标记设置type属性, 不能省略

```
1 <script type="module" src="./js/main.js"></script>
```

- 必须在服务器环境下打开, 不然解析不了会报错, 在vscode里面安装 Live Server 插件, 这个插件就可以提供服务器环境



4、单个导出和导入示例代码

```
1 export let index = 2
2 export let cart = ()=>{
3     console.log('购物车功能')
4 }
5
6 import { index, cart } from './cart.js'
```

5、默认导出和导入示例代码

```
1 let index = 2
2 function cart(){
3     console.log('购物车功能')
4 }
5
6 // 注意点: 如果对象的键和值一样的情况下, 可以省略值
7 export default {
8     index,
9     cart
10 }
11
12 import cartMod from './cart.js'
13 console.log(cartMod.index)
```

六、node环境安装

- 1、下载地址: <https://nodejs.cn/>
- 2、下载版本, 需要选择长期支持版本

长期支持版本: **24.14.1**



3、什么是node?

- node 是一个后端运行环境，基于Chrome v8引擎的运行环境
- node 后端开发所使用的语言是 JavaScript

4、注意点

- 前端项目如果使用框架开发（vue.js、react.js）那么必须使用 node来作为环境
- 如果不使用 node，项目是启动不了

5、验证是否安装成功

```
Last login: Fri Apr 10 13:14:13 on ttys001
dingwei@dwdeMacBook-Air ~ % node -v
v22.19.0
dingwei@dwdeMacBook-Air ~ %
```

6、电脑安装了 node环境，那么会自动安装 npm 包管理器（和python pip是类似的东西）

7、npm 镜像源地址默认是在国外，下载安装很多东西的时候很慢甚至报错，需要切换镜像源

8、推荐大家使用 pnpm，基于 npm做了优化的工具

地址：<https://www.pnpm.cn/>

七、前端框架vue.js

1、框架可以帮助咱们可以快速进行开发，配套的有很多相关的功能库

2、介绍，作者：尤雨溪

渐进式 JavaScript 框架

易学易用，性能出色，适用场景丰富的 Web 前端框架。

[快速上手 →](#)[安装](#)[获取针对 Vue 2 的安全更新 !\[\]\(8b57f0e15e7dda24cf9977561475f640_img.jpg\)](#)

3、组件概念

把页面不同的区域称之为组件，一个组件是包含三个部分：

- 结构
- 样式
- 行为